

AIR UNIVERSITY
Department of Electrical
College of Engineering

ARTIFICIAL INTELLIGENCE LAB

LAB02

Lab Objective: Data Preliminaries: Fetching data using Pandas library (Python), checking for anomalies in data and data imputation using scikit-learn library (Python)

Class ID: _____

Lab Date: _____

Student's ID: _____

Lab 02	Data Preliminaries: Fetching data using Pandas library (Python), checking for anomalies in data and data imputation using scikit-learn library (Python)				
PLO's	PLO3 – Design and Development		Bloom's Taxonomy	C3 – Apply	
LAB TASK PERFORMANCE					
CLO's	Aspects of Assessments	Excellent (75-100%)	Average (50-75%)	Poor (<50%)	M
CLO2 100%	<u>Design & Development</u> Design solution for problem that meets specified needs by using the concept of artificial intelligence.	A complete solution / Explain necessary requirements according to task description with great use of time and resource material.	Solution was complete but need minor modifications /student could have followed specification more closely.	Solution was complete but did not work, needed several modifications / did not make correct use of resource material or instructions.	
Total Marks: 10					

Graded by Engineer: _____

Date: _____

Remarks: _____

Objective:

1. Make attendee familiar with the concept of fetching data files using Pandas specifically in a cloud-computing environment (Google Co-laboratory for example)
2. Make attendee understand how to check the anomalies in data
3. Make attendee familiar with the concept of data imputation using Scikit-learn (Python)

Importing Data in a Cloud-computing Environment

pandas is a fast, powerful, flexible and easy to use open source data analysis and manipulation tool, built on top of the Python programming language. It can be used to fetch and manipulate most types of standard datasets and therefore is a very important tool for AI-experts and data scientists.

A dataset called 'train_new.csv' is provided to students and students will fetch this data through their Google co-lab environment as follows:

For Google Colab

```
>>import pandas as pd
>>import numpy as np
>>from google.colab import files
>>uploaded=files.upload()
>>import io
>>df1=pd.read_csv(io.BytesIO(uploaded['train_new.csv']))
```

For Spyder (the above commands a modified a little bit to reduce the extra steps of uploading data to cloud environment as data will be run on your computer instead of cloud)

```
import pandas as pd
import numpy as np
df1=pd.read_csv('E:/Courses/ArtificialIntelligence/Fall2021/Lab/AI Lab
Manuals/train_new.csv')
(note:you will use the path where you have put the dataset)
print (df1.shape)
```

Once the data is acquired, it is always instructive to note the dimensions of the data. Students are encouraged to note that there are missing values in the data. In the following code we will define the missing values present in the data and ask the python to treat them as missing rather than assuming a garbage value. Also it is to be noted that the data fetched gives *n* rows while the actual dimension is of *n+1* rows (value of *n* is depending on the dataset given to you by your teacher). This is because by default pandas take the first row as a header and doesn't fetch it. So if there is no header, as in this case, we will have to give an explicit command within the pd.read_csv command:

```
>>df1.shape
>>missing_values=["n/a", "NAN", "na", " "]
>>df1=pd.read_csv(io.BytesIO(uploaded['train_new.csv']),na_values=missing_val
ues,header=None)
>>#df1.shape
>>a=df1.isnull()
>>#print(a)
>>df1.shape
```

(you can easily modify above code for spyder as guided above)

Data Imputation:

For various reasons, many real world datasets contain missing values, often encoded as blanks, NaNs or other placeholders. Such datasets however are incompatible with AI model-development which assumes that all values in an array are numerical, and that all have and hold meaning. A basic strategy to use incomplete datasets is to discard entire rows and/or columns containing missing values. However, this comes at the price of losing data which may be valuable (even though incomplete). A better strategy is to impute the missing values, i.e., to infer them from the known part of the data.

The following snippet demonstrates how to replace missing values, encoded as `np.nan`, using the mean value of the columns (axis 0) that contain the missing values:

```
>>>
>>>import numpy as np
>>>from sklearn.impute import SimpleImputer
>>>imp = SimpleImputer(missing_values=np.nan, strategy='mean')
>>>imp.fit([[1, 2], [np.nan, 3], [7, 6]])
SimpleImputer()
>>>X = [[np.nan, 2], [6, np.nan], [7, 6]]
>>>print(imp.transform(X))
[[4.         2.         ]
 [6.         3.666...]
 [7.         6.         ]]
```

Lab Task:

(CLO2, Marks 4) Complete hands-on activities as prescribed above.

(CLO 6, Marks 1) Perform properly handling of lab.

Home Task:

4. (CLO2, Marks 2) The dataset is provided in the file `train_new`. Write a code to automatically find the location of the missing values and replace it by zero.
5. (CLO2, Marks 3) Write a program to perform basic data imputation using the 'mean' strategy. You may use `sklearn.impute` with `SimpleImputer`